

## 28/8/26 DAY 5:

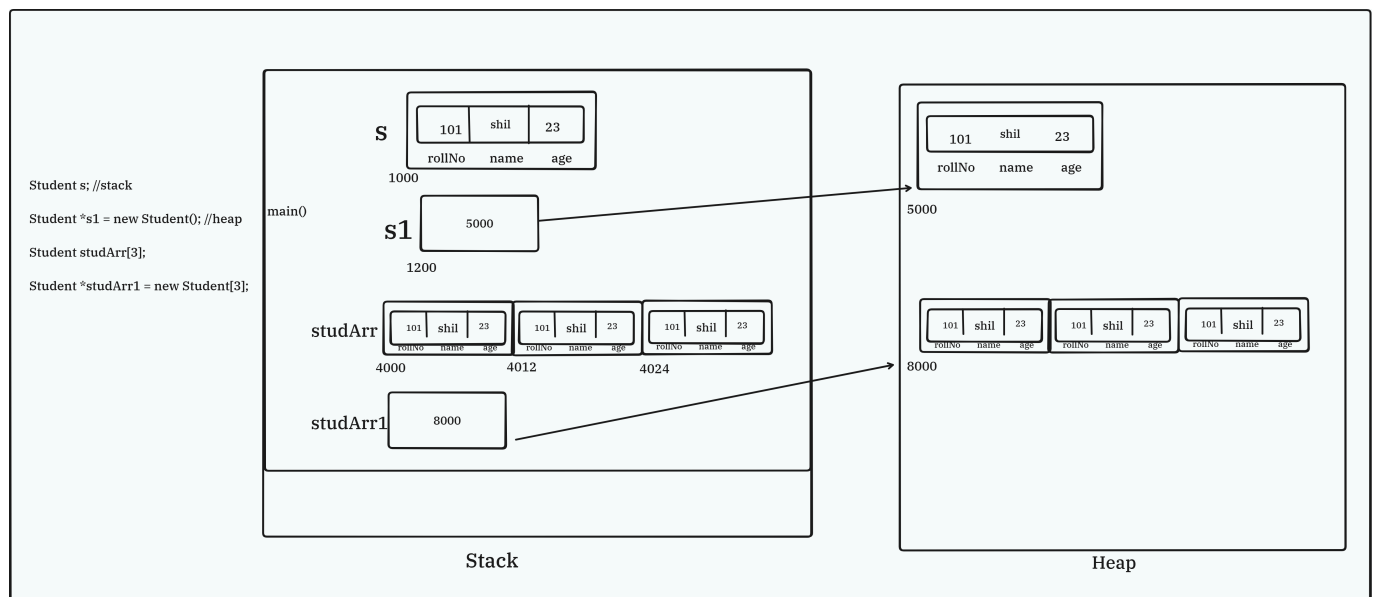
### Array of Objects

- In order to have multiple objects of a class, we can create an array of objects.
- It is similar to creating an array of primitive data types.
- It is also possible to create an array of objects dynamically using the new operator.

Syntax:

```
// stack memory allocation
ClassName objectArray[size];

// heap memory allocation
ClassName* objectArray = new ClassName[size];
```



### Function Overloading & Default Arguments

#### Function Overloading

- Function overloading is a feature in C++ that allows you to have multiple functions with the same name but different parameter lists (different types or numbers of parameters).
- The compiler determines which function to call based on the arguments passed to the function.

Rules for Function Overloading:

1. Functions must have the same name.
2. Functions must have different parameter lists (either in number or type of parameters).
3. Return type alone is not sufficient to distinguish overloaded functions.

```
#include <iostream>
using namespace std;

// c language
int addInt(int a, int b){
    return a + b;
}

double addDouble(double a, double b){
    return a + b;
}

//Name Mangling
//Z9_addii
int add(int a, int b){           //add(int,int)
    return a + b;
}

double add(int a, double b){     //add(int,double)
    return a + b;
}

double add(double b, int a){      //add(double,int)
    return a + b;
}

//int add(double a, int b){       //add(double,int)
//    return a + b;
//}

float add(float a, float b){
    cout << "float" << endl;
    return a + b;
}

double add(double a, double b){ //add(double,double)
    return a + b;
}

int add(int a, int b, int c){    //add(int,int,int)
    return a + b + c;
}

double getFarenheit(int temp = 37){ // default argument
    return (temp * 1.8) + 32;
}

int subtract(int a = 5, int b){ //default
    return a - b;
}

int main() {
```

```

int a = 10;
int b = 10;
int c = 20;

cout << addInt(a,b) << endl;
cout << add(10.5f,23.3f) << endl;
cout << add(a,b,c) << endl;
cout << add(a,b) << endl;

// cout << subtract(a, b) << endl;
cout << subtract(a,b) << endl;

cout << getFarenheit(40) << endl;

return 0;
}

```

## Default Arguments

- Default arguments are values that are assigned to parameters in the function declaration.
- If a function is called without providing a value for a parameter with a default argument, the default value is used.

### Rules for Default Arguments:

1. Default arguments must be specified in the function declaration, not in the function definition.
2. Default arguments must be specified from right to left.
3. If a function has multiple parameters with default arguments, all parameters to the right of the first default argument must also have default arguments.

```

subtract(int a = 5, int b = 10) {
    return a - b;
}
// subtract(int a, int b = 10) {    // valid
//     return a - b;
// }

// subtract(int a = 10, int b) {    // invalid
//     return a - b;
// }

int main() {
    int a = 10;
    int b = 20;

    cout << subtract(a, b) << endl; // Calls subtract with both arguments
    cout << subtract(b) << endl;    // Calls subtract with default value for 'a'
    cout << subtract() << endl;    // Calls subtract with default values for
    both 'a' and 'b'
}

```

```
    return 0;
}
```

When using the function overloading and default arguments together, it must be ensured that the function calls are unambiguous.

## Constructor Concept

- A constructor is a special member function of a class that is automatically called when an object of that class is created.
- It is used to initialize the object's data members and allocate resources if needed.
- A constructor has the same name as the class and does not have a return type, not even void.
- C++ provides a default constructor if no constructors are explicitly defined in the class. However, if any constructor is defined, the default constructor is not provided automatically.
- Constructors can be overloaded, meaning a class can have multiple constructors with different parameter lists

## Default Constructor

A default constructor is a constructor that can be called with no arguments. It is either explicitly defined by the programmer or automatically provided by the compiler if no other constructors are defined.

```
class MyClass {
public:
    MyClass() { // Default constructor
                // Initialize data members
    }
};
```

## Parameterized Constructor

A parameterized constructor is a constructor that takes parameters to initialize an object with specific values. It allows you to create objects with different initial states.

```
class MyClass {
public:
    MyClass(int a, int b) { // Parameterized constructor
                            // Initialize data members with the provided values
    }
};
```

## Multiple Constructors

A class can have multiple constructors, each with a different set of parameters. This allows for creating objects in different ways. The concept of having multiple constructors is known as constructor overloading.

## Constructor Overloading

```
class MyClass {  
public:  
    MyClass() { // Default constructor  
        // Initialize data members  
    }  
  
    MyClass(int a) { // Parameterized constructor with one parameter  
        // Initialize data members with the provided value  
    }  
  
    MyClass(int a, int b) { // Parameterized constructor with two parameters  
        // Initialize data members with the provided values  
    }  
};
```

## Dynamic Initialization of Objects

- Dynamic initialization of objects refers to the process of creating and initializing objects at runtime, rather than at compile time.
- This is typically done using dynamic memory allocation, allowing for more flexible and efficient use of resources.

```
int* ptr = new int(5); // Dynamic initialization
```

## Constructor Initializer List

- A constructor initializer list is a way to initialize member variables of a class before the constructor's body executes.
- It is specified after the constructor's parameter list and before the constructor's body, using a colon (😊) followed by a comma-separated list of member initializations.
- The initializer list is useful for initializing const and reference members, as well as members that don't have default constructors.

```
class MyClass {  
private:  
    int a;  
    int b;  
public:  
    MyClass(int x, int y) : a(x), b(y) { // Constructor initializer list  
        // Constructor body  
    }  
};
```

## Destructor

- A destructor is a special member function of a class that is automatically called when an object of that class goes out of scope or is explicitly deleted.
- It is used to release resources that the object may have acquired during its lifetime, such as dynamically allocated memory, file handles, or network connections.

```
class MyClass {  
public:  
    ~MyClass() { // Destructor  
                // Release resources  
    }  
};
```

## Object Lifecycle

- The lifecycle of an object refers to the sequence of events that occur from the time an object is created until it is destroyed. It includes the following stages:
  1. **Creation:** The object is created, and its constructor is called to initialize its data members.
  2. **Usage:** The object is used in the program, and its member functions can be called to perform operations on its data members.
  3. **Destruction:** The object goes out of scope or is explicitly deleted, and its destructor is called to release any resources it may have acquired.

## Dynamic Memory

- It is nothing but the memory which is allocated at runtime using new operator. It is also called heap memory.
- Dynamic memory is managed by the programmer, who is responsible for allocating and deallocating memory as needed.

### new operator

- The new operator is used to dynamically allocate memory for an object or an array of objects.
- It returns a pointer to the allocated memory, which can be used to access the object(s).

```
int* ptr = new int(5); // Allocate memory for an integer
```

### delete operator

- The delete operator is used to deallocate memory that was previously allocated using the new operator.

```
delete ptr; // Deallocate the memory
```

## Dynamic variables

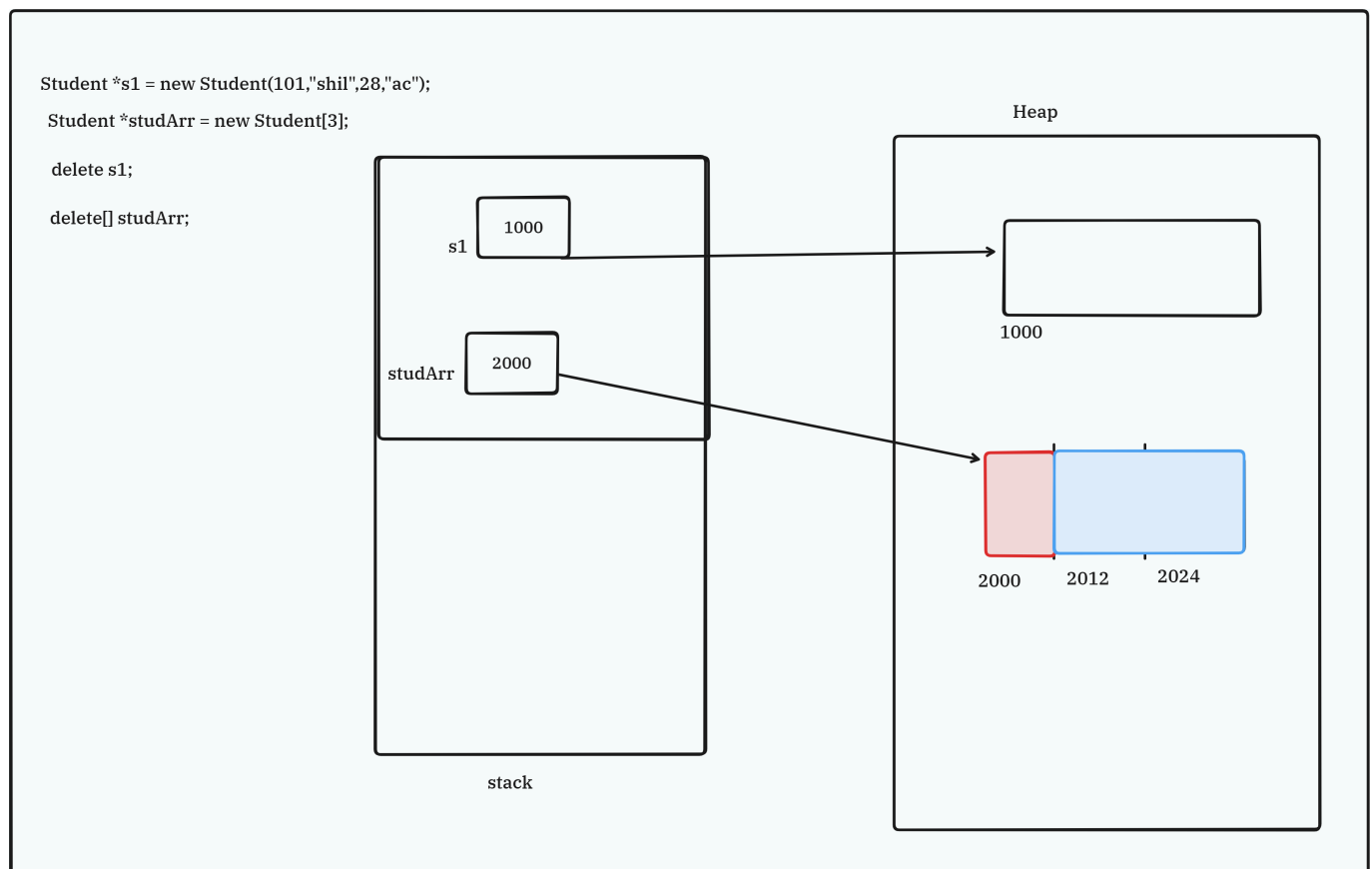
- Dynamic variables are variables that are created at runtime and can be resized as needed.
- They are typically created using the new operator and can be accessed through pointers.

```
int* dynamicVar = new int(10); // Create a dynamic variable
```

## Dynamic arrays

- Dynamic arrays are arrays whose size is determined at runtime, rather than at compile time.
- They are typically created using the new operator and can be resized as needed.

```
int* dynamicArray = new int[size]; // Create a dynamic array
```



## Tomorrow's Topics:

### Namespaces

### std Namespace

### Different Scopes

### Header files

## **Header guards**

**#include < > vs " "**

## **Dynamic 1-D arrays**

## **Dynamic 2-D arrays**

## **Pointer to Objects**

## **Class Pointer**

## **this Pointer**

## **Enumeration**

## **typedef**

## **Type Aliases**

## **using type aliases**

## **Resource Ownership Concept**

## **Memory Leak**

## **Dangling Pointer**

## **Double Deletion**

## **RAII Concept**

## **Introduction to Smart Pointers**

## **Why Smart Pointers?**